armaan_lab1

January 18, 2026

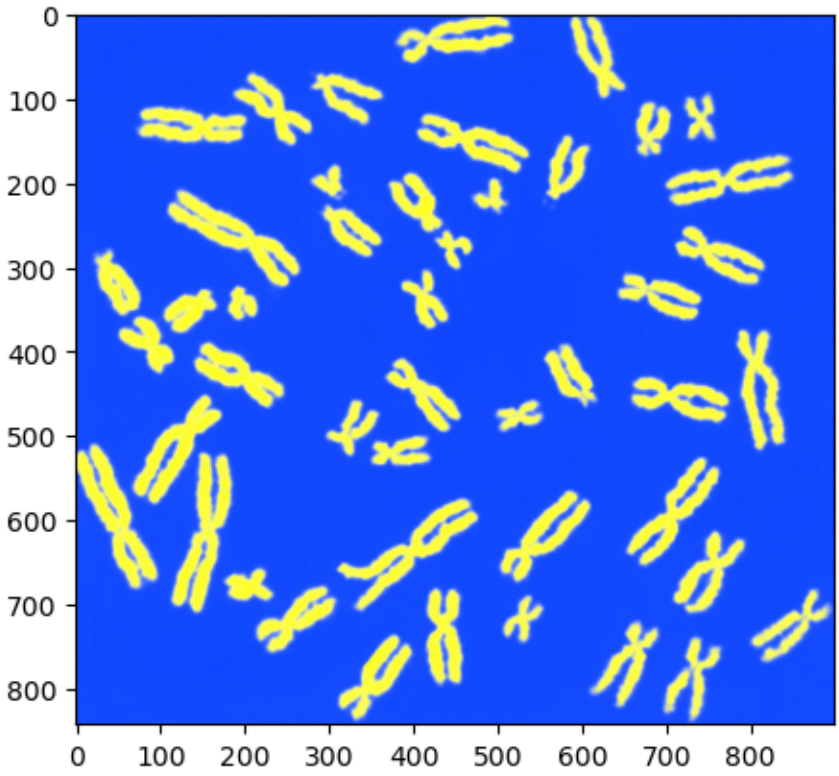
[1]:
```
%pip install opencv-python numpy matplotlib
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: opencv-python in
/home/armaan/.local/lib/python3.14/site-packages (4.12.0.88)
Requirement already satisfied: numpy in /home/armaan/.local/lib/python3.14/site-
packages (2.2.6)
Requirement already satisfied: matplotlib in /usr/lib64/python3.14/site-packages
(3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in /usr/lib64/python3.14/site-
packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/lib/python3.14/site-packages
(from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /usr/lib64/python3.14/site-
packages (from matplotlib) (4.61.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/lib64/python3.14/site-
packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/lib/python3.14/site-
packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/lib64/python3.14/site-packages
(from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=3 in /usr/lib/python3.14/site-packages
(from matplotlib) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/lib/python3.14/site-
packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.14/site-packages
(from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

[3]:
```
image = cv2.imread('chromosomes.jpg')
# the image was loaded in RGB, but matplotlib uses BGR internally. Fix found on␣
 ↪stackoverflow
plt.imshow(image[..., ::-1])
```

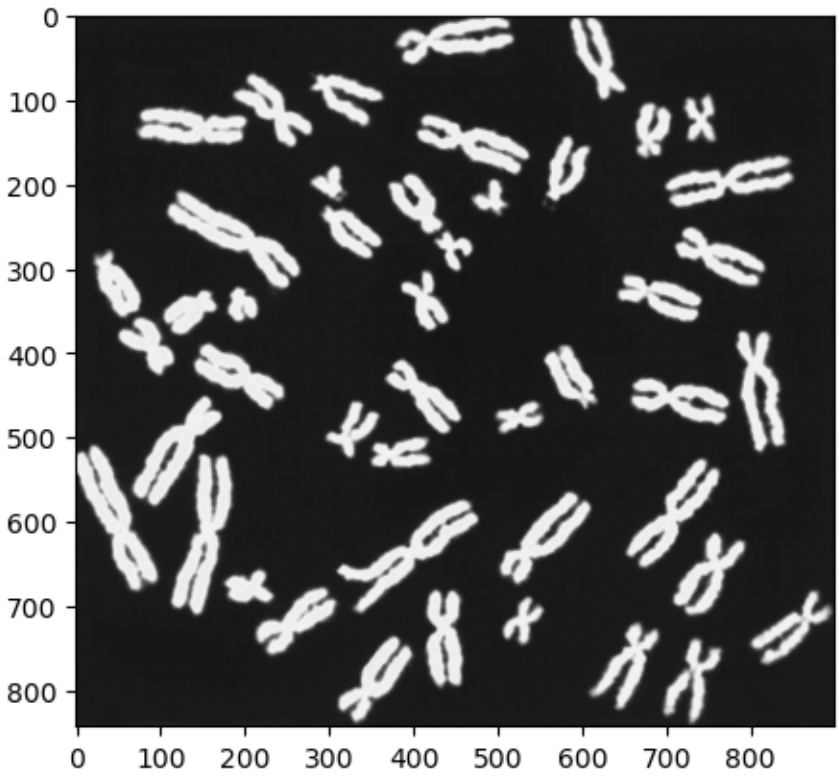
[3]: <matplotlib.image.AxesImage at 0x7f0958583380>



[4]:
```
grayscale_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

#need cmap="gray" to display grayscale images correctly, found solution on
 ↪stackoverflow
plt.imshow(grayscale_image, cmap="gray")
```

[4]: <matplotlib.image.AxesImage at 0x7f099036ccd0>



[5]:
```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
opening = cv2.morphologyEx(grayscale_image, cv2.MORPH_ELLIPSE, kernel)
kernel
```

[5]:
```
array([[1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1]], dtype=uint8)
```

[6]:
```
_, dst = cv2.threshold(opening, 170, 255, cv2.THRESH_BINARY)
```

[7]:
```
contours, hierarchy = cv2.findContours(dst, cv2.RETR_EXTERNAL, cv2.
↪CHAIN_APPROX_SIMPLE)
```

[8]:
```
len(contours)
```

[8]:
```
46
```

[9]:
```
#calculate all the needed values and store ina df
import pandas as pd
info =[]
```

```
for i in contours:
    Area= cv2.contourArea(i)
    Perimeter = cv2.arcLength(i, True)
    Circularity = (4* np.pi* Area) / (Perimeter**2) if Perimeter else None
    x, y, w, h = cv2.boundingRect(i)
    image = cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
    info.append((h,w,i.shape, Area, Perimeter, Circularity))
df = pd.DataFrame(info, columns=['Height', 'Width', 'Contour Shape', 'Area',␣
 ↪'Perimeter', 'Circularity'])
df
```

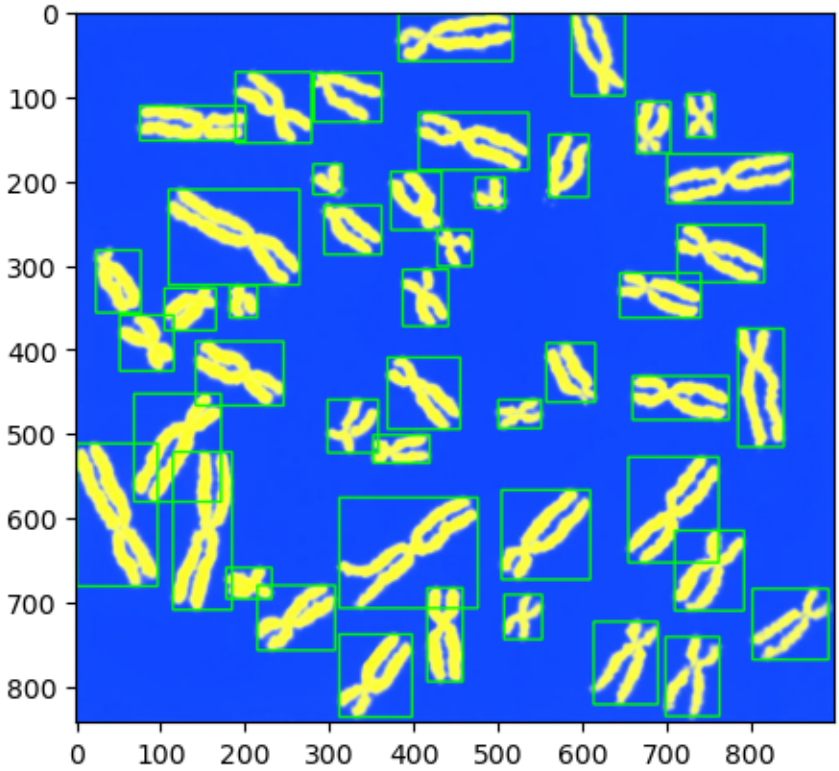
[9]:

	Height	Width	Contour Shape	Area	Perimeter	Circularity
0	94	64	(144, 1, 2)	1924.0	410.735062	0.143315
1	98	86	(218, 1, 2)	2889.5	446.457930	0.182168
2	98	76	(207, 1, 2)	2298.0	428.617311	0.157188
3	53	45	(86, 1, 2)	901.0	209.681239	0.257523
4	84	90	(190, 1, 2)	2003.5	433.002086	0.134282
5	111	42	(162, 1, 2)	2996.0	470.048770	0.170399
6	77	92	(158, 1, 2)	2740.0	383.504614	0.234110
7	37	54	(74, 1, 2)	1237.0	177.681239	0.492375
8	95	82	(186, 1, 2)	2678.0	456.274165	0.161647
9	131	164	(258, 1, 2)	5170.0	605.896530	0.176972
10	106	105	(229, 1, 2)	3338.0	536.097540	0.145952
11	125	107	(242, 1, 2)	3673.5	597.452879	0.129325
12	187	70	(256, 1, 2)	5417.5	686.374671	0.144506
13	169	96	(245, 1, 2)	5596.0	556.215290	0.227301
14	33	67	(97, 1, 2)	1366.5	262.409161	0.249380
15	34	51	(63, 1, 2)	885.0	189.539104	0.309568
16	63	60	(125, 1, 2)	1350.0	264.450791	0.242579
17	128	103	(240, 1, 2)	4141.0	542.298550	0.176945
18	52	114	(193, 1, 2)	2823.5	485.546243	0.150500
19	85	86	(155, 1, 2)	2350.5	385.688379	0.198562
20	70	58	(100, 1, 2)	2110.0	222.308656	0.536512
21	76	104	(138, 1, 2)	3227.0	342.190905	0.346315
22	140	54	(234, 1, 2)	3330.5	592.717816	0.119131
23	66	64	(114, 1, 2)	1860.0	241.137083	0.401971
24	50	61	(62, 1, 2)	1699.0	185.338094	0.621547
25	38	33	(46, 1, 2)	830.0	127.740115	0.639195
26	53	96	(164, 1, 2)	2304.5	402.291411	0.178939
27	67	54	(115, 1, 2)	1456.5	267.379723	0.256014
28	74	53	(83, 1, 2)	2078.0	211.480228	0.583870
29	43	41	(66, 1, 2)	754.0	171.539103	0.321999
30	68	103	(200, 1, 2)	2563.5	439.060962	0.167107
31	58	68	(96, 1, 2)	1996.0	227.279219	0.485569
32	113	155	(226, 1, 2)	5073.0	534.298550	0.223309
33	36	35	(45, 1, 2)	638.5	129.497474	0.478463
34	69	60	(119, 1, 2)	1874.0	287.563488	0.284782

35	36	34	(48, 1, 2)	666.5	121.982755	0.562877
36	58	148	(251, 1, 2)	3285.0	625.587874	0.105480
37	74	47	(122, 1, 2)	1636.0	308.592926	0.215884
38	68	130	(231, 1, 2)	3277.5	543.144223	0.139612
39	41	125	(126, 1, 2)	3623.5	387.865005	0.302675
40	61	40	(85, 1, 2)	1405.5	258.894442	0.263509
41	51	33	(85, 1, 2)	908.5	215.580734	0.245649
42	58	80	(136, 1, 2)	1812.0	332.249780	0.206271
43	84	90	(164, 1, 2)	2598.5	344.291410	0.275474
44	97	63	(173, 1, 2)	2228.0	398.333041	0.176454
45	56	135	(225, 1, 2)	3206.5	564.717816	0.126351

[10]:
```
plt.imshow(image[..., ::-1])
```

[10]:
```
<matplotlib.image.AxesImage at 0x7f08e97d8b90>
```



1 Cleaning and Normalizing

[11]:
```
df.dropna(inplace=True)
```

[12]:
```
df_standardized = df.copy()
df_normalized = df.copy()
```

[13]:
```
for col in ['Height', 'Width', 'Area', 'Perimeter', 'Circularity']:
    df_standardized[col] = (df[col] - df[col].mean()) / df[col].std()
    df_normalized[col] = (df[col] - df[col].min()) / (df[col].max() - df[col].
 ↪min())
```

[14]:
```
for col in ['Height', 'Width', 'Area', 'Perimeter', 'Circularity']:
    df_standardized = df_standardized[df_standardized[col].abs() < 3]
```

[15]:
```
df_standardized
```

[15]:

	Height	Width	Contour	Shape	Area	Perimeter	Circularity
0	0.467374	-0.432034	(144, 1, 2)	-0.407292	0.264707	-0.849364
1	0.580677	0.216658	(218, 1, 2)	0.355428	0.495738	-0.585687
2	0.580677	-0.078202	(207, 1, 2)	-0.111841	0.380357	-0.755211
3	-0.693980	-0.992268	(86, 1, 2)	-1.215435	-1.035571	-0.074285
4	0.184117	0.334602	(190, 1, 2)	-0.344489	0.408715	-0.910664
5	0.948912	-1.080726	(162, 1, 2)	0.439560	0.648307	-0.665560
6	-0.014163	0.393574	(158, 1, 2)	0.237327	0.088599	-0.233180
7	-1.147192	-0.726894	(74, 1, 2)	-0.950004	-1.242525	1.519560
8	0.495700	0.098714	(186, 1, 2)	0.188349	0.559223	-0.724950
9	1.515426	2.516565	(258, 1, 2)	2.156963	1.526877	-0.620951
10	0.807283	0.776892	(229, 1, 2)	0.709731	1.075465	-0.831471
11	1.345472	0.835864	(242, 1, 2)	0.974768	1.472269	-0.944308
13	2.591803	0.511518	(245, 1, 2)	2.493492	1.205573	-0.279385
14	-1.260495	-0.343576	(97, 1, 2)	-0.847702	-0.694563	-0.129546
15	-1.232169	-0.815352	(63, 1, 2)	-1.228074	-1.165837	0.278921
16	-0.410723	-0.549978	(125, 1, 2)	-0.860737	-0.681359	-0.175699
17	1.430449	0.717920	(240, 1, 2)	1.344080	1.115569	-0.621132
18	-0.722306	1.042266	(193, 1, 2)	0.303290	0.748534	-0.800602
19	0.212443	0.216658	(155, 1, 2)	-0.070368	0.102722	-0.474423
20	-0.212443	-0.608950	(100, 1, 2)	-0.260356	-0.953906	1.819097
21	-0.042489	0.747406	(138, 1, 2)	0.622044	-0.178590	0.528314
22	1.770358	-0.726894	(234, 1, 2)	0.703807	1.441646	-1.013493
23	-0.325746	-0.432034	(114, 1, 2)	-0.457850	-0.832136	0.906024
24	-0.778957	-0.520492	(62, 1, 2)	-0.585036	-1.193006	2.396191
25	-1.118866	-1.346099	(46, 1, 2)	-1.271523	-1.565510	2.515962
26	-0.693980	0.511518	(164, 1, 2)	-0.106707	0.210099	-0.607599
27	-0.297420	-0.726894	(115, 1, 2)	-0.776604	-0.662417	-0.084525
28	-0.099140	-0.756380	(83, 1, 2)	-0.285636	-1.023936	2.140492
29	-0.977237	-1.110212	(66, 1, 2)	-1.331561	-1.282248	0.363290
30	-0.269094	0.717920	(200, 1, 2)	0.097897	0.447899	-0.687901
31	-0.552352	-0.314090	(96, 1, 2)	-0.350413	-0.921759	1.473368
32	1.005563	2.251191	(226, 1, 2)	2.080336	1.063831	-0.306477
33	-1.175517	-1.287127	(45, 1, 2)	-1.422803	-1.554145	1.425141

34	-0.240769	-0.549978	(119, 1, 2)	-0.446790	-0.531882	0.110710
35	-1.175517	-1.316613	(48, 1, 2)	-1.400684	-1.602745	1.998021
36	-0.552352	2.044789	(251, 1, 2)	0.667863	1.654227	-1.106136
37	-0.099140	-0.933296	(122, 1, 2)	-0.634804	-0.395878	-0.356866
38	-0.269094	1.514041	(231, 1, 2)	0.661938	1.121038	-0.874496
39	-1.033889	1.366611	(126, 1, 2)	0.935269	0.116799	0.232146
40	-0.467374	-1.139698	(85, 1, 2)	-0.816893	-0.717294	-0.033659
41	-0.750632	-1.346099	(85, 1, 2)	-1.209510	-0.997417	-0.154867
42	-0.552352	0.039742	(136, 1, 2)	-0.495769	-0.242882	-0.422107
43	0.184117	0.334602	(164, 1, 2)	0.125546	-0.165005	0.047541
44	0.552352	-0.461520	(173, 1, 2)	-0.167140	0.184499	-0.624461
45	-0.609003	1.661471	(225, 1, 2)	0.605850	1.260562	-0.964492

[16]:
```
df_normalized
```

[16]:

	Height	Width	Contour	Shape	Area	Perimeter	Circularity
0	0.396104	0.236641	(144, 1, 2)	0.259304	0.511617	0.070890
1	0.422078	0.404580	(218, 1, 2)	0.454060	0.574911	0.143687
2	0.422078	0.328244	(207, 1, 2)	0.334745	0.543301	0.096885
3	0.129870	0.091603	(86, 1, 2)	0.052950	0.155386	0.284877
4	0.331169	0.435115	(190, 1, 2)	0.275340	0.551070	0.053967
5	0.506494	0.068702	(162, 1, 2)	0.475542	0.616710	0.121636
6	0.285714	0.450382	(158, 1, 2)	0.423903	0.463369	0.241009
7	0.025974	0.160305	(74, 1, 2)	0.120726	0.098688	0.724910
8	0.402597	0.374046	(186, 1, 2)	0.411397	0.592304	0.105239
9	0.636364	1.000000	(258, 1, 2)	0.914070	0.857407	0.133951
10	0.474026	0.549618	(229, 1, 2)	0.544528	0.733736	0.075830
11	0.597403	0.564885	(242, 1, 2)	0.612204	0.842447	0.044678
12	1.000000	0.282443	(256, 1, 2)	0.963994	1.000000	0.073122
13	0.883117	0.480916	(245, 1, 2)	1.000000	0.769381	0.228252
14	0.000000	0.259542	(97, 1, 2)	0.146848	0.248810	0.269620
15	0.006494	0.137405	(63, 1, 2)	0.049723	0.119698	0.382391
16	0.194805	0.206107	(125, 1, 2)	0.143520	0.252427	0.256878
17	0.616883	0.534351	(240, 1, 2)	0.706505	0.744723	0.133902
18	0.123377	0.618321	(193, 1, 2)	0.440746	0.644168	0.084353
19	0.337662	0.404580	(155, 1, 2)	0.345335	0.467238	0.174405
20	0.240260	0.190840	(100, 1, 2)	0.296823	0.177759	0.807608
21	0.279221	0.541985	(138, 1, 2)	0.522138	0.390169	0.451244
22	0.694805	0.160305	(234, 1, 2)	0.543016	0.834057	0.025577
23	0.214286	0.236641	(114, 1, 2)	0.246394	0.211120	0.555524
24	0.110390	0.213740	(62, 1, 2)	0.213918	0.112254	0.966933
25	0.032468	0.000000	(46, 1, 2)	0.038628	0.010201	1.000000
26	0.129870	0.480916	(164, 1, 2)	0.336056	0.496656	0.137638
27	0.220779	0.160305	(115, 1, 2)	0.165003	0.257617	0.282050
28	0.266234	0.152672	(83, 1, 2)	0.290368	0.158573	0.896339
29	0.064935	0.061069	(66, 1, 2)	0.023298	0.087805	0.405684
30	0.227273	0.534351	(200, 1, 2)	0.388301	0.561805	0.115468

31	0.162338	0.267176	(96, 1, 2)	0.273828	0.186566	0.712157
32	0.519481	0.931298	(226, 1, 2)	0.894503	0.730549	0.220772
33	0.019481	0.015267	(45, 1, 2)	0.000000	0.013315	0.698843
34	0.233766	0.206107	(119, 1, 2)	0.249218	0.293379	0.335951
35	0.019481	0.007634	(48, 1, 2)	0.005648	0.000000	0.857005
36	0.162338	0.877863	(251, 1, 2)	0.533838	0.892297	0.000000
37	0.266234	0.106870	(122, 1, 2)	0.201210	0.330639	0.206861
38	0.227273	0.740458	(231, 1, 2)	0.532325	0.746222	0.063952
39	0.051948	0.702290	(126, 1, 2)	0.602118	0.471095	0.369477
40	0.181818	0.053435	(85, 1, 2)	0.154715	0.242583	0.296093
41	0.116883	0.000000	(85, 1, 2)	0.054463	0.165839	0.262629
42	0.162338	0.358779	(136, 1, 2)	0.236712	0.372555	0.188849
43	0.331169	0.435115	(164, 1, 2)	0.395361	0.393891	0.318511
44	0.415584	0.229008	(173, 1, 2)	0.320625	0.489643	0.132982
45	0.149351	0.778626	(225, 1, 2)	0.518003	0.784446	0.039106

1.0.1 Q1: How can contour detection be used to identify objects in an image?

Contour detection marks the boundary of each disjoint object in the image. This can detect objects in the image by giving them their individual contours. Unrequired objects can then be filtered out with other tests like color etc.

1.0.2 Q2: What is the importance of standardization of data? What difference did you observe before and after standardization?

Each category had its own range of values and units. It was hard to understand the outliers and general distribution. Standardization reduced all values to a standard gaussian distribution. It was also easy to remove the outliers by checking for a standard deviation threshold (3 from the mean).

1.0.3 Q3: Let's consider one of the values in the width column is missing. How to handle this missing value?

In the code, I have dropped such values during cleaning, but we can also replace them by the mean or median or with a regressed value based on other columns.

1.0.4 Q4: What is the importance of data normalization? What difference did you observe before and after normalization?

Each category had its own range of values and units and it was thus hard to understand the general distribution. Normalization reduced all values to a [0,1] range. This helped me identify the extremity of these values.

1.0.5 Q5: How might you adapt the bounding box construction process to handle overlapping or touching chromosomes?

In a previous version of my code, I was using a different method to finding contours. It used the RETR_TREE method instead of the RETR_EXTERNAL method. This resulted in my finding contours within other contours. I realised this when I plotted the bounding boxes and realized that I should instead be using RETR_EXTERNAL.

[ ]: